

Static Analysis and Code Quality Review

Project and Evidence

Project: Task Manager FastAPI, a REST API for user registration/authentication and task CRUD, with WebSocket notifications and SQLAlchemy persistence.

Project statistics: Before refactoring there were 17 application Python files and 441 source lines; after refactoring the same 17 files contain 574 lines. The test suite has 4 Python files and 228 lines. The supplied baseline Pylint output reports 450 analyzed lines and 237 statements. The report is the UTF-16 file `reports/pylint_original_report.txt`; its original Pylint 4.0.8 run reports 48 convention findings, 3 refactor findings, 5 warnings, 48 errors, and a score of 0.00/10. Several of those errors are analyzer setup artifacts, especially package discovery and Alembic's dynamic API.

Controlled comparison: Pylint 4.0.8, application package `app`, the committed `.pylintrc`, and identical invocation before/after refactoring. This excludes the Alembic scaffold (migration scripts are generated and use dynamic Alembic APIs), and sets the project source root. Before changes: 7.69/10, 43 messages (40 conventions, 3 warnings). After changes: 10.00/10, no messages. Full outputs are in `reports/pylint_before.txt` and `reports/pylint_after.txt`.

Test baseline and verification: The initial unmodified suite had 8 passing and 3 failing tests. The failures were caused by the tests sharing a configured PostgreSQL database, duplicate persistent test records, and an unbound `Session()` in model tests. After replacing that setup with per-test in-memory SQLite and adding regression coverage, `pytest -q` reports **13 passed**. The captured final output is `reports/test_results.txt`. Tests verify task ownership isolation and WebSocket notification delivery in addition to prior endpoint/model behavior.

Baseline reference: The original repository state is commit `bda7ad7` (Baseline before static analysis). The final history is intentionally progressive: baseline/configuration, API fixes, isolated tests, and report evidence are separate commits. The original report was preserved byte-for-byte.

Part 4: Meaningful Finding Categories

Pylint's categories are implementation groupings; the quality concern depends on what the message says and the surrounding code.

Quality concern	Representative finding	Software-quality meaning
Naming and readability	<code>invalid-name</code>	Names that do not communicate intent or follow local conventions make code harder to scan. Framework-mandated names can be valid exceptions.
Documentation	<code>missing-function-docstring</code> , <code>missing-module-docstring</code>	The API and intent are harder to discover and maintain, especially for public or non-obvious behavior. Boilerplate may have low documentation value.
Complexity and comprehension	<code>too-many-branches</code>	Many decision paths increase cognitive load and the number of cases to test. No such warning was present in this baseline.
Function design	<code>too-many-arguments</code>	A broad parameter list can signal mixed responsibilities or poor grouping; it can also accurately express a small framework boundary. Not reported in this baseline.

Quality concern	Representative finding	Software-quality meaning
Class design	too-many-instance-attributes, too-few-public-methods	May indicate a class with too many responsibilities or a simple data holder that does not need methods. ORM/Pydantic models are intentionally declarative data holders.
Maintainability	duplicate-code	Changes must be repeated consistently and can drift. Pylint reported no duplicated lines here.
Error handling	raise-missing-from	Re-raising without chaining loses causal context needed for diagnosis.
Unused/dead code	unused-import	Unused names add noise and can conceal real dependencies; imports can be intentional for registration side effects.
Possible defect / analyzer mismatch	relative-beyond-top-level, no-member, import-error	Could represent broken imports or missing members, but package roots and dynamic framework APIs can mislead static inference. Verify by running the application/tool in its proper context.
Coding convention	line-too-long, logging-fstring-interpolation	Formatting and logging practices reduce consistency or defer formatting unnecessarily; usually lower risk than access-control defects.

Parts 5-6: Ten Findings and Assessment

Line numbers below refer to the **original baseline source**, matching the supplied Pylint report. Each classification assesses the reported code, not the report's authority.

1. Task endpoint lacks a module docstring

A. Tool finding: app/api/endpoints/tasks.py, line 1; C0114, missing-module-docstring.

B. Original code:

```
from typing import List, Set

from fastapi import APIRouter, Depends, HTTPException, WebSocket, WebSocketDisconnect
```

C. Explanation: The module begins with imports and gives no one-line description of the module's responsibility. Pylint reports this because module-level intent is not documented.

D. Quality impact: Small reduction in discoverability and maintainability for contributors navigating endpoint code; it does not change runtime reliability.

Part 6 decision: B - Context dependent. A module with self-explanatory route names needs less prose than a complex module. Here a concise module docstring is inexpensive and now present, but this finding alone is not evidence of a serious design flaw.

2. Task creation endpoint lacks a function docstring

A. Tool finding: app/api/endpoints/tasks.py, line 30; C0116, missing-function-docstring.

B. Original code:

```
def create_task(task: TaskCreate, db: Session = Depends(get_db), username: str = Depends(
```

```
get_user_by_token)):
    user = db.query(User).filter(User.username == username).first()
    db_task = Task(**task.dict(), owner_id=user.id)
```

C. Explanation: The endpoint's contract and authorization intent are undocumented; its name says it creates a task but not who owns it, what is persisted, or what happens on failure. The rule reports the missing docstring, not the separate security defect in this code.

D. Quality impact: Modifiability and testability are affected because maintainers must infer intent; reliability/security are affected by the actual unchecked user lookup and missing ownership checks, which are human-review findings rather than the docstring diagnostic.

Part 6 decision: B - Context dependent. Trivial helpers can reasonably omit docstrings; public API operations with authorization and persistence deserve a short contract. A concise docstring was added while the actual ownership defect was fixed independently.

3. Overlong task creation line

A. Tool finding: `app/api/endpoints/tasks.py`, line 30; C0301, line-too-long (109 characters versus configured 100).

B. Original code:

```
def create_task(task: TaskCreate, db: Session = Depends(get_db), username: str = Depends(
    get_user_by_token)):
```

C. Explanation: Several dependencies and the body model are packed into one line, forcing horizontal scrolling and hiding the endpoint's parameter structure.

D. Quality impact: Readability and modifiability; a reviewer can more easily overlook dependency changes in a dense signature. It has little direct effect on runtime behavior.

Part 6 decision: A - Should be fixed. Wrapping the signature is a low-cost consistency improvement. The project retains the 100-character limit and the final endpoint signature is multiline.

4. Relative-import error from Pylint package discovery

A. Tool finding: `app/api/endpoints/tasks.py`, line 7; E0402, relative-beyond-top-level (the same error was emitted for the adjacent relative imports).

B. Original code:

```
from ..models.task import TaskCreate, TaskUpdate, TaskResponse
from ...core.security import get_user_by_token
from ...db.database import get_db
```

C. Explanation: When Pylint was invoked without the repository source root, it inferred the module as top-level `api.endpoints.tasks`; climbing to the `app` package then appeared to go beyond the package. The application imports these modules successfully as `app.api.endpoints.tasks`.

D. Quality impact: If real, this would harm reliability and deployability. In this instance the diagnostic was a tooling configuration error, not a code defect; the Pylint source-root hypothesis was confirmed by rerunning with `--source-roots=.` and observing these errors disappear.

Part 6 decision: C - Should not be fixed / acceptable exception. The relative imports are valid within the project's package. Changing them to work around an incorrect module root would obscure the real fix; configure Pylint with `source-roots=.`

5. Eager f-string interpolation in logging

A. Tool finding: `app/api/middleware/middleware.py`, line 15; W1203, logging-fstring-interpolation (also line 17).

B. Original code:

```
logger.info(f"Incoming request: {request.method} {request.url.path}")
response = await call_next(request)
logger.info(f"Outgoing response code: {response.status_code}")
```

C. Explanation: F-strings build their messages before the logger checks whether INFO is enabled. Logging's parameterized formatting defers interpolation and is consistent with its API.

D. Quality impact: Minor performance under disabled logging and consistent operational logging; no meaningful correctness issue at this traffic level.

Part 6 decision: B - Context dependent. It is worth adopting because the lazy form is idiomatic and essentially free, but it should not be ranked alongside authorization defects. Both calls now use `%s` parameters.

6. JWT exception is raised without causal chain

A. Tool finding: `app/core/security.py`, line 37; W0707, raise-missing-from.

B. Original code:

```
except JWTError:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate token",
        headers={"WWW-Authenticate": "Bearer"},
    )
```

C. Explanation: A JWT decoding failure is converted to the API's 401 response, but the original `JWTError` is not explicitly linked to the new exception. That makes the causal relationship less clear in traceback/debugging contexts.

D. Quality impact: Operability and maintainability for diagnosing malformed, expired, or otherwise invalid tokens. The client-facing error remains appropriately generic.

Part 6 decision: A - Should be fixed. Preserve the sanitized response while chaining the cause using `except JWTError as exc and raise ... from exc`. This change was made.

7. SessionLocal flagged as an invalid constant name

A. Tool finding: `app/db/database.py`, line 12; C0103, invalid-name.

B. Original code:

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

C. Explanation: Pylint treats the uppercase middle as inconsistent with its configured variable naming regex. SQLAlchemy documentation and common applications conventionally call this factory `SessionLocal`.

D. Quality impact: Naming consistency, but renaming affects readability only and can be less recognizable to SQLAlchemy developers.

Part 6 decision: C - Should not be fixed / acceptable exception. `SessionLocal` is a widely understood framework idiom, so the style rule is a false positive for this project's conventions. In this implementation it was renamed to `session_factory` because that more directly describes its role, not because Pylint's style rule is inherently correct; no behavior changes.

8. Alembic model imports reported unused

A. Tool finding: `app/alembic/env.py`, line 12; W0611, unused-import for `User` and `Task`.

B. Original code:

```
from app.db.database import Base
from app.db.db_structure import User, Task

target_metadata = Base.metadata
```

C. Explanation: Alembic needs model modules imported so their tables register in `Base.metadata`; the imported names are not subsequently referenced by name, so ordinary unused-import analysis flags them.

D. Quality impact: Removing them can cause migrations/autogeneration to miss model tables, harming migration reliability. Keeping them without context can confuse maintainers.

Part 6 decision: C - Should not be fixed / acceptable exception. The imports have a registration side effect and are meaningful to Alembic. A comment or a targeted Pylint suppression could make the intent clearer if this file is included in routine linting. This scaffold is excluded from the configured application score.

9. Alembic operation API reported as missing members

A. Tool finding: `app/alembic/versions/001_dd43bed97b33_initial.py`, line 23; E1101, no-member for `alembic.op.create_table` (the report contains similar findings for other operations).

B. Original code:

```
from alembic import op

def upgrade():
    op.create_table(
        "user",
```

C. Explanation: Alembic exposes migration operations through a runtime-provided proxy. Pylint cannot infer the members from the static import in this execution context.

D. Quality impact: A real missing migration operation would affect deployability/data integrity, but this is a known dynamic API and the generated migration is valid framework code. The diagnostic itself is not evidence that the operation will fail.

Part 6 decision: C - Should not be fixed / acceptable exception. Do not rewrite a valid migration to satisfy inference. Exclude migration scaffolding or configure a narrowly scoped inference exception, and still execute migration checks when migration behavior is in scope.

10. Task schema lacks a class docstring

A. Tool finding: app/api/models/task.py, line 4; C0115, missing-class-docstring.

B. Original code:

```
class TaskCreate(BaseModel):
    title: str
    description: str
```

C. Explanation: The schema does not explain that these are fields required to create a task. Pylint reports the missing class-level description.

D. Quality impact: Discoverability and maintainability for API/schema users; type fields themselves remain explicit and validation is unaffected.

Part 6 decision: B - Context dependent. A two-field model is self-explanatory and repeated docstrings can become noise, but its request/response role is a useful contract. Short schema docstrings were added to distinguish the types' API roles.

Part 7: Independent Manual Review

These observations come from tracing behavior and tests, not simply restating Pylint messages. Items 1-4 were not directly reported by Pylint.

1. Task records were not scoped to their owner (not reported by Pylint)

Original code:

```
# create_task
user = db.query(User).filter(User.username == username).first()
db_task = Task(**task.dict(), owner_id=user.id)

# read_tasks
return db.query(Task).offset(skip).limit(limit).all()
```

Problem: Creation assumes the user lookup always succeeds, and list/read/update/delete queries used only task IDs or no owner condition. Any authenticated account could potentially enumerate, read, modify, or delete another account's task.

Quality impact: Security and reliability; this is a cross-account confidentiality and integrity failure, far more serious than a style finding.

Recommendation and change: Resolve the authenticated user once, use a shared get_owned_task_or_404 lookup for single-task operations, and filter list queries by owner_id. Return the same 404 for absent and foreign-owned IDs to avoid confirming another user's record exists. Regression tests exercise list, read, update, and delete using a second account.

2. Async WebSocket sends were not awaited (not reported by Pylint)

Original code:

```
for connection in active_connections:
    connection.send_text(f"New task created: {db_task.title}")
```

Problem: `send_text` is asynchronous. Calling it without `await` creates a coroutine but does not deliver the notification. Dead connections also remained in the set, and removing an already-removed socket could raise an error.

Quality impact: Reliability and user-visible correctness; event notifications could silently never be delivered and stale clients could destabilize the handler.

Recommendation and change: Add `ConnectionManager` with awaited broadcast, discard on disconnect, and stale-connection cleanup. Schedule async notifications through FastAPI BackgroundTasks; a WebSocket regression checks the exact task-created message.

3. Tests depended on persistent PostgreSQL and global mutable state (not reported by Pylint)

Original code:

```
client = TestClient(app)
jwt_token = None

# tests/test_models.py
db = Session()
```

Problem: Endpoint tests reused a module-global client/token and real configured database rows, producing duplicate-key failures on later runs. `Session()` had no engine bind, so model persistence tests could not commit.

Quality impact: Testability and reliability: outcomes depended on execution order, database contents, and local credentials rather than the code under test.

Recommendation and change: Add per-test in-memory SQLite engines/sessions and FastAPI dependency overrides. Register/login as needed per test; clear overrides and dispose each engine. The suite now passes without touching PostgreSQL.

4. Router-wide HTTP authentication conflicted with the WebSocket route (not reported by Pylint)

Original code:

```
app.include_router(
    tasks.router,
    prefix="/api/v1",
    tags=["Tasks"],
    dependencies=[Depends(get_user_by_token)],
)
```

Problem: `get_user_by_token` depends on `OAuth2PasswordBearer`, which expects an HTTP Request. The same router included a WebSocket route; its scope did not satisfy that HTTP dependency, so the socket handshake failed before endpoint logic ran.

Quality impact: Reliability and testability of the real-time feature; a user-visible feature was unreachable despite HTTP endpoints working.

Recommendation and change: Put user resolution on each HTTP operation through `get_current_user` and validate the Bearer token from WebSocket headers in the WebSocket handler. The endpoint test connects and receives an event.

5. Importing middleware configured a file handler as a side effect

Original code:

```
handler = logging.FileHandler('info.log')
handler.setLevel(logging.INFO)
logger.addHandler(handler)
```

Problem: Importing the module opens/creates a working-directory file and mutates the global logger. Reloads or multiple app instances can attach duplicate handlers; test and production process setup is coupled to import order.

Quality impact: Modifiability and operational reliability; duplicate log lines, unexpected file creation, and hard-to-isolate tests are possible.

Recommendation: Configure logging once at the application composition/startup boundary, or use standard logging configuration; make handler creation idempotent and inject/use the configured logger. This assignment kept the logger setup otherwise unchanged to avoid expanding scope beyond the targeted refactors.

Part 8: Pylint Configuration Decisions

The committed `.pylintrc` was generated with Pylint 4.0.8 and then reviewed rather than accepted blindly.

Setting	Generated default	Decision and rationale
max-line-length	100	Retain. It is a practical limit for endpoint signatures and readable diffs; wrap long signatures and exception construction.
max-args	5	Retain. It is a useful prompt to inspect function responsibilities; framework dependency parameters sometimes make a warning context-dependent, so do not mechanically bundle unrelated values.
max-branches	12	Retain. Branch counts are a useful complexity signal; no current function required an exception.
source-roots	empty	Change to <code>..</code> . This correctly models app as a package and removes false <code>relative-beyond-top-level</code> errors.
ignore-paths	empty	Set to <code>^app[/\\]alembic[/\\]</code> . Generated migration scripts and Alembic's dynamic proxy API otherwise dominate with known inference noise; migrations still require migration/runtime validation when changed.
too-few-public-methods	enabled	Disable for this project. Declarative SQLAlchemy models and settings/Pydantic classes are useful data structures even when they intentionally expose no methods. Keep design review for actual class responsibility problems.

The 100-character limit, five-argument threshold, and 12-branch threshold are retained. Disabling `too-few-public-methods` is a scoped tradeoff, not a claim that all data-holder classes are well-designed.

Parts 9-10: Refactors and Verification

At least five significant improvements were made; three address structural/code-quality behavior rather than cosmetic formatting.

Improvement 1: Centralize authorized task lookup (structural/security)

Before:

```
task = db.query(Task).filter(Task.id == task_id).first()
if task is None:
    raise HTTPException(status_code=404, detail="Task not found")
```

After:

```
def get_owned_task_or_404(db: Session, task_id: int, owner_id: int) -> Task:
    task = db.query(Task).filter(
        Task.id == task_id,
        Task.owner_id == owner_id,
    ).first()
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return task
```

Why better: One helper provides consistent not-found behavior and makes ownership an explicit part of every single-task operation. List queries also filter on owner ID.

Improvement 2: Separate connection lifecycle and await broadcasts (structural/reliability)

Before:

```
active_connections.add(websocket)
...
for connection in active_connections:
    connection.send_text(message)
```

After:

```
class ConnectionManager:
    async def broadcast(self, message: str):
        disconnected = set()
        for connection in tuple(self.active_connections):
            try:
                await connection.send_text(message)
            except (RuntimeError, WebSocketDisconnect):
                disconnected.add(connection)
        self.active_connections.difference_update(disconnected)
```

Why better: Connection ownership is cohesive, asynchronous sends execute, iteration is stable, and broken clients are cleaned up.

Improvement 3: Isolate database tests (structural/testability)

Before:

```
client = TestClient(app)
db = Session()
```

After:

```
engine = create_engine(
    "sqlite://",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
Base.metadata.create_all(bind=engine)
```

Why better: Each test gets a bound disposable database and overrides `get_db`, avoiding persistent test contamination and external PostgreSQL dependency.

Improvement 4: Use current Pydantic serialization API

Before:

```
db_task = Task(**task.dict(), owner_id=user.id)
```

After:

```
db_task = Task(**task.model_dump(), owner_id=current_user.id)
```

Why better: Uses Pydantic v2's supported API and makes the authenticated owner explicit. This also removes runtime deprecation warnings from this call site.

Improvement 5: Preserve JWT error cause

Before:

```
except JWTError:
    raise HTTPException(status_code=401, detail="Could not validate token")
```

After:

```
except JWTError as exc:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate token",
        headers={"WWW-Authenticate": "Bearer"},
    ) from exc
```

Why better: The client still receives a generic authentication response, while server diagnostics retain the original decode failure.

Verification evidence: `python -m pytest -q --tb=short` completed with 13 passed. Pylint after the changes returned 10.00/10 and no findings. A focused WebSocket test confirmed the expected broadcast message, and an ownership regression confirmed a second user receives an empty list/404 and cannot alter the owner's record. The full application was imported through `TestClient`; production startup against PostgreSQL was not run because it requires the configured external service. Remaining runtime warnings are from the pinned `python-jose` implementation's internal `datetime.utcnow()` use, not this project's changed code.

Part 11: Repeated Static Analysis

Metric	Before (controlled config)	After	Change
Pylint score	7.69/10	10.00/10	+2.31
Total findings	43	0	-43
Convention issues	40	0	-40
Warnings	3	0	-3
Refactoring findings	0	0	0
Errors	0	0	0

Did the score improve? Yes, 7.69 to 10.00 under identical Pylint/config/package scope. The supplied unconfigured report says 0.00/10; that figure is not a fair before measure for this comparison because it includes package-root false errors and Alembic dynamic API noise.

Which categories improved most? Convention findings (mostly missing docs and line length) and warnings (logging formatting and exception chaining) went to zero. Package path configuration removed the relative-import errors before the controlled baseline. The manual security and WebSocket fixes are much more important than their direct Pylint score contribution.

Did new warnings appear? No Pylint messages appeared after the refactor. Runtime tests still show 14 warnings from python-jose internals using deprecated UTC time APIs.

Does the number represent code quality fully? No. Pylint cannot infer authorization policy or guarantee async coroutine execution here; those defects required manual review and regression tests. Exclusions and disabled messages also affect scores, so the committed config and original output must accompany the score.

Part 12: Static Analysis Versus Human Review

- **What did Pylint detect well?** Consistent, mechanically checkable issues: missing descriptions, overlong lines, eager log interpolation, weak exception chaining, and naming differences. Proper source-root configuration also exposed which import errors were real versus package-discovery artifacts.
- **What needed human judgment?** Whether each API query was scoped to its owner, whether async sends were awaited, whether a WebSocket route could use an HTTP dependency, why Alembic imports exist for side effects, and whether test state was isolated. These required following call paths and executing tests.
- **False positives or low-value issues?** Alembic's `op.create_table/context.configure` members are dynamic, and migration model imports are registration side effects. `SessionLocal` is a familiar SQLAlchemy naming idiom despite Pylint's preference. Missing docstrings on obvious data schemas can be low priority. The original relative-import errors resulted from the wrong source root. Each requires contextual interpretation.
- **Can a high score coexist with poor design?** Yes. Before manual fixes, the project had a 7.69 configured score while task list/detail/update/delete operations lacked owner checks and notification coroutines were never awaited; neither defect was reported by Pylint. A score could be made perfect by suppressing messages while leaving both defects intact.
- **Should static analysis replace review?** It should complement human review. Automated analysis gives fast, repeatable coverage for patterns; people assess product security boundaries, framework interactions, tests, operational assumptions, and whether a warning is an acceptable convention. This project required both: Pylint cleaned up measurable issues, while manual investigation found and tests verified the consequential defects.

Submission Contents

- `reports/assignment_report.pdf`: rendered report.
- `reports/assignment_report.md`: editable source for the PDF.
- `reports/pylint_original_report.txt`: supplied baseline output, preserved unchanged.
- `reports/pylint_before.txt` and `reports/pylint_after.txt`: controlled before/after runs.
- `reports/test_results.txt`: final test evidence.
- `.pylintrc`: generated and reviewed project configuration.

- Source and tests, including SQLite fixtures and new regressions.
- Git history with separate commits for baseline/configuration, application fixes, test isolation, and report/package evidence.